UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

PROGRAMACIÓN I

Unidad Temática N°4:
Diseño de Formularios con C#

Material Teórico
1° Año – 1° Cuatrimestre

CC BY ND NC

Índice

DISEÑO DE FORMULARIOS CON C# 2

Programación Visual 2
El modelo de programación Windows 2
Entornos de programación Visual 3
Componentes 3
Propiedades, métodos y eventos 4
Componentes visuales y no visuales 5
Componentes básicos 5
Primer Proyecto Windows Form 9
Problema modelo 16

BIBLIOGRAFÍA 18

DISEÑO DE FORMULARIOS CON C#

Programación Visual

La mayor parte de las soluciones necesitan ofrecer al usuario un modo de interactuar con la aplicación. En el ejemplo de aplicación comercial, un sitio Web permite al cliente ver productos y realizar pedidos, y una aplicación de escritorio basada Windows permite a los representantes de ventas escribir los datos de los pedidos de los clientes que se han contactado con la empresa de manera telefónica. Las interfaces de usuario se implementan utilizando formularios de Windows Forms, páginas Microsoft ASP.NET, controles u otro tipo de tecnología que permita procesar y dar formato a los datos de los usuarios, así como adquirir y validar los datos entrantes procedentes de éstos.

El modelo de programación Windows

El modelo de programación propuesto por Windows es totalmente diferente al modelo de ejecución secuencial de DOS basado en caracteres. Al ser Windows un entorno multitarea los programas tienen que estar preparados para compartir los recursos del equipo (procesador, memoria, etc.). Esto supone que Windows ha de disponer de métodos que permitan suspender tareas para activar otras en función de las circunstancias del momento (por ejemplo, por acción del usuario). Este hecho supone que las aplicaciones han de cooperar en la compartición de esos recursos. Las aplicaciones Windows se limitan a “esperar” mensajes procedentes del sistema, procesarlos y volver al estado de espera. Este modelo de programación se conoce como “orientado al evento”.

Mensaje. Es una notificación a la aplicación de que ha ocurrido algo de interés y que por lo tanto debe de realizarse alguna acción específica. El origen del mensaje puede ser el usuario (haciendo click con el ratón dentro de una ventana), la propia aplicación (mandándose un mensaje a si misma) o Windows (pidiendo, por ejemplo, que se despliegue la ventana tras ocultarse otra que tuviese delante). Dado que la unidad mínima de ejecución en Windows es una ventana, los mensajes van realmente dirigidos a ellas.

Ventana y procedimiento de ventana. En Windows, una aplicación se representa físicamente por su ventana principal (aunque después pueda desplegar diversas ventanas hijas). Cada una de esas ventanas dispone de una serie de propiedades y un código asociado (lo que concuerda con el principio de la POO, en el concepto de objeto). Al código asociado a cada ventana se le denomina **procedimiento de ventana**, y es el responsable de recibir los mensajes, procesarlos y devolver el control a Windows para quedar en espera.

Orientado a eventos. Esto significa que cualquier programa está condicionado por los eventos que ocurren en el entorno Windows (movimiento del ratón, pulsación de algún botón del ratón, pulsación de una tecla del teclado, desplazamiento o redimensionamiento de una ventana, etc.). Un programa Windows está continuamente sondeando al sistema ante la ocurrencia de cualquier evento y de ocurrir, Windows avisa al programa enviándole un mensaje. Un programa Windows está ocioso la mayor parte del tiempo esperando a que ocurra algún evento.

Otra de las características específicas de Windows frente a DOS es el uso de recursos por parte de las aplicaciones, como son iconos, menús, mapas de bits, cursores, plantillas de diálogos, etc. Las aplicaciones Windows disponen por tanto de recursos (gráficos generalmente) propios almacenados en lo que se llama el fichero de recursos. El proceso de construcción de programas en Windows incorpora una fase adicional al compilado y enlazado de los módulos objeto y las librerías. Hay un proceso final de compilación y de enlazado (bind) del fichero de recursos.

Entornos de programación Visual

En estos entornos, la labor de un programador se parece más a la de un “ensamblador” de piezas de software que la de un “constructor” de software. Con esto conseguimos mayor rapidez de desarrollo y, sobre todo, mayor simplicidad, ya que sólo tenemos que saber cómo “montar” esas piezas para que nuestro programa funcione. Delphi fue uno de los primeros entornos en aplicar con éxito esta filosofía, y hoy en día son muchos los que apuestan por esta idea, como por ejemplo Microsoft con su lenguaje de programación C#.

Estos entornos Se conocen con el nombre de **IDEs**. Los IDEs son una herramienta esencial a la hora de desarrollar software ya que incluyen:

- Editor (herramientas y opciones de desarrollo)
- Compilador o intérprete
- Depurador
- Ayuda en línea

Componentes

Para explicar esto debemos conocer la programación orientada a objetos, ya que la programación basada en componentes se apoya sobre ella.

Un componente es una clase de uso específico, lista para usar, que puede ser configurada o utilizada de forma visual, desde el entorno de desarrollo.

La principal diferencia, respecto a una clase normal, es que la mayor parte del trabajo lo podemos hacer de forma visual, con el ratón y ajustando las opciones que se nos ofrece en nuestro entorno.

En la programación orientada a objetos, debemos codificar una serie de operaciones, más o menos laboriosas, para preparar los objetos para su uso. Programar estas operaciones requiere su tiempo, su complejidad y pueden ser origen de errores. Sin embargo, en la programación basada en componentes, todas estas operaciones las realizamos de forma visual, para así poder dedicar la atención a nuestro problema. Nuestro trabajo se centrará en programar las respuestas a los **eventos** disparados por usuario sobre las pantallas.

Propiedades, métodos y eventos

Las propiedades son datos públicos del componente, muy parecidas a los atributos de una clase, aunque se accede a ellas a través de dos métodos: un método para leer su valor, y otro para modificarlo. Existen propiedades de sólo lectura, en las que podemos consultar, pero no modificar su valor, y propiedades de sólo escritura. Por ejemplo, las propiedades “Alto” (Width) y “Ancho” (Height) de un botón permiten que un programador pueda cambiar las dimensiones del componente. Cuando el programador cambia alguna de ellas, el componente debe redibujarse en la pantalla, para mostrar los nuevos cambios.

Los métodos permiten realizar acciones específicas sobre un componente. Normalmente, se utilizan métodos para dos tareas distintas: realizar algo importante (como repintar en pantalla o cambiar el foco), o para establecer el valor de los atributos internos, haciendo algún tipo de comprobación previa.

Los métodos de eventos son funciones del componente, que se ejecutarán automáticamente en respuesta a un evento. **Un evento es** un suceso que ocurre desde el exterior del componente y que puede condicionar el comportamiento y apariencia del programa, por ejemplo: movimiento del ratón, pulsación de algún botón del ratón, pulsación de una tecla del teclado, desplazamiento o redimensionamiento de una ventana, etc. Un programador puede poner el código que quiera en el evento, para así poder hacer una acción cuando ese “algo importante” ocurra. Cada componente poseerá una serie de eventos que puede recibir o generar. Se pueden tratar los eventos de un componente que necesitemos, y dejar que los demás sean tratados por defecto.

Por último, **los atributos**. Tienen la misma misión que en programación orientada a objetos, es decir: almacenar datos internos al objeto (o clase). En el maravilloso mundo de los componentes, los atributos siempre son internos y de uso

privado, y debemos utilizar las propiedades para que un programador pueda leer o establecer un dato.

Sabiendo esto, podemos decir que la principal “misión” del programador de componentes es definir un grupo de propiedades, métodos y eventos para que otros programadores puedan utilizar el componente de forma sencilla y rápida.

Componentes visuales y no visuales

Se pueden establecer muchas clasificaciones para los componentes. Una de ellas es la de visuales o controles, frente a no visuales.

Los componentes visuales son aquellos que, al utilizarlos, muestran algún elemento (o dibujo) en la pantalla y es el usuario de nuestros programas el que interactúa con él. El componente es el principal responsable de dibujar en la pantalla lo que sea oportuno, dependiendo de su estado, del valor de sus atributos, etc. Hay muchos componentes de este tipo, como pueden ser los botones, etiquetas, cajas de texto, formas, etc. Estos componentes suelen ser llamados **controles**.

Los componentes no visuales son aquellos que no aparecen en la ventana, y se insertan en un formulario para que el programador los utilice. Son más fáciles de programar que los componentes visuales, ya que no tienen ningún tipo de interfaz gráfico. Ejemplos de componentes no visuales podrían ser un temporizador, un contenedor, una librería (componente ddl) o una conexión a base de datos.

Componentes básicos

Formularios o Ventanas

Imagen 1: Elaboración propia

En la mayoría de aplicaciones, el formulario es la parte de la interfaz que permite que el usuario pueda introducir datos u obtener datos de su interés. Su diseño es crítico dado que una mala concepción de él puede convertirlo en una barrera para la interacción e inducir al usuario a cometer errores potenciando la frustración en el desempeño de su tarea.

Actualmente los formularios, al menos como estilo de interacción, los podemos encontrar en las aplicaciones más diversas. Utilizamos formularios cuando en una página web introducimos nuestros datos de tarjeta de crédito para efectuar un pago, pero también cuando en un cajero automático indicamos la cantidad de dinero que

deseamos retirar o cuando en un editor seleccionamos las opciones de formato del texto.

La siguiente lista de directrices de diseño son consejos para realizar un buen diseño de formularios:

- Dar un título al formulario que exprese claramente su función.
- Las instrucciones han de ser breves y comprensibles.
- Hacer grupos lógicos de campos y separarlos con blancos. Por ejemplo: apellido, primer nombre y segundo nombre es un grupo lógico.
- Aspecto ordenado alineando los campos y las etiquetas.
- Las etiquetas de los campos deben usar terminología familiar al usuario.
- Ser consistente en el uso de los términos, es decir, usar siempre las mismas palabras para los mismos conceptos.
- El tamaño visible del campo debe corresponderse con la longitud del contenido que ha de introducir el usuario.
- Permitir el movimiento del cursor por medio del teclado y no solo con el mouse.
- Permitir que el usuario pueda corregir con libertad los caracteres que ha introducido en los campos.
- En donde sea posible, impedir que el usuario introduzca valores incorrectos. Por ejemplo, impedir que introduzca caracteres alfabéticos en campos que solo admiten valores numéricos.
- Si introduce valores incorrectos, indicar en un mensaje cuales son los correctos.
- Avisar cuanto antes al usuario si ha introducido valores incorrectos. Si es posible, no esperar a que haya rellenado el formulario totalmente. Esto no debe tomarse tan literal, ya que muchas veces las validaciones inmediatas pueden ser vividas por el usuario como un control excesivo y frustrante.
- Marcar claramente los campos obligatorios o los opcionales.
- Si es posible, colocar explicaciones o la lista de los valores válidos al lado de los campos.
- Dejar clara la acción que debe hacer el usuario al terminar de rellenar el formulario.

Ventanas de Diálogos

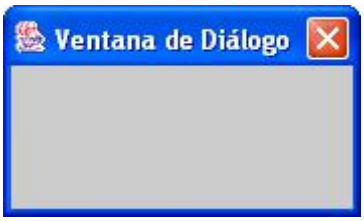

Imagen 2: Elaboración propia

Una ventana de diálogo es muy parecida a una ventana con la diferencia de que no puede ser minimizada ni maximizada. Se utiliza cuando se requiere una respuesta o interacción precisa del usuario, de allí su nombre de ventana de "diálogo".

Ofrece el mismo comportamiento que una ventana, agregando la capacidad de tomar el control del foco para una aplicación hasta ser cerrada. Cuando una ventana de diálogo posee esta capacidad se dice que es una ventana modal (se convierte en una ventana exclusiva en el sentido de que deja inhabilitada cualquier operación con el resto de la aplicación). Para lograr que nuestro formulario se comporte de manera modal solo es necesario ejecutar el método **showDialog(),** indicando opcionalmente el formulario padre como parámetro, al momento de poner visible nuestro formulario.

Diálogos con opción

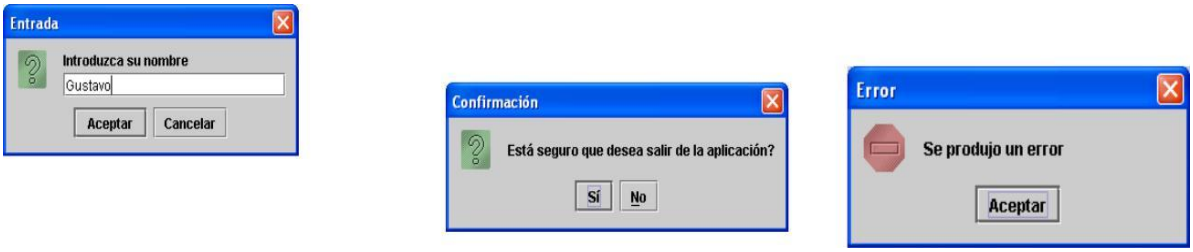

Imagen 3: Elaboración propia

Son diálogos prefabricados, que tienen los componentes básicos para permitir, por ejemplo, mostrar un mensaje, o pedir algún valor o confirmación al usuario.

Controles

A continuación, se listan los controles más frecuentes utilizados para diseñar nuestras aplicaciones basadas en formularios:

MenuStrip: Barra de menú (que actúa como menú principal).

PopupMenu: Menú desplegable (también llamado menú contextual) que aparecen cuando se presiona con el botón derecho del ratón.

Label: Muestra texto que el usuario no puede seleccionar ni manipular. Se usa para mostrar textos de título, encabezamientos, o incluso para mostrar resultados.

TextBox: Muestra un área de edición de texto en la que el usuario puede introducir y modificar una única línea de texto.

Memo: Muestra un área de edición de texto en la que el usuario puede introducir y modificar múltiples líneas de texto.

Button: Crea un botón que el usuario puede presionar para efectuar acciones.

CheckBox: Presenta una opción binaria (Si/No - Verdad/Falso) de manera que cuando se selecciona este control, se permuta entre ambos valores. Este control puede emplearse para crear un grupo de estos controles que representen elecciones que no sean mutuamente exclusivas (al contrario que los RadioButton), por lo que el usuario puede seleccionar más de una opción en un grupo.

RadioButton: Presenta una opción binaria (Si/No - Verdad/Falso) de manera que cuando se selecciona este control, se permuta entre ambos valores. Este control puede emplearse para crear un grupo de estos controles que representen elecciones mutuamente exclusivas (al contrario que los CheckBox), por lo que el usuario puede seleccionar sólo una en un grupo.

ScrollBar: Proporciona una forma cómoda de modificar el área visible de un formulario o de una lista. También puede usarse para desplazarse en un rango amplio de valores por incrementos prefijados.

GroupBox: Contenedor para agrupar opciones relacionadas en un formulario.

Panel: Contenedor que puede contener otros componentes en un formulario. Se usa para crear barras de herramientas y líneas de estado. Los componentes que contiene están asociados al panel.

ListBox: Muestra una lista de elecciones que está acompañada de una barra de scroll.

ComboBox: Muestra una lista de elecciones. Es un control que combina aspectos de un componente ListBox y de un componente TextBox: el usuario puede introducir datos en el área de edición o seleccionar en el área de lista.

Primer Proyecto Windows Form

Vamos a crear un proyecto de Aplicación Windows en C# paso a paso. Primero seleccionar la opción **Crear nuevo proyecto.**

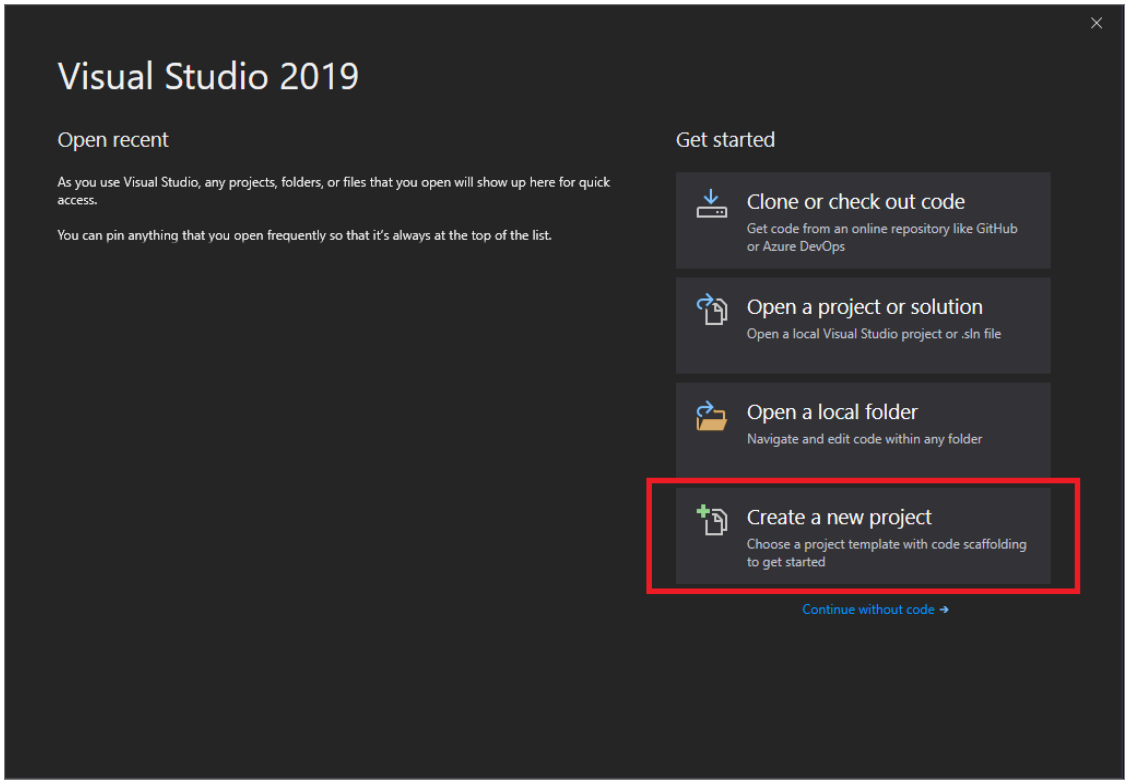

Imagen 4: Elaboración propia

En la ventana Crear un nuevo proyecto, elegir la plantilla Aplicación de Windows Forms (.NET Framework) para C#.

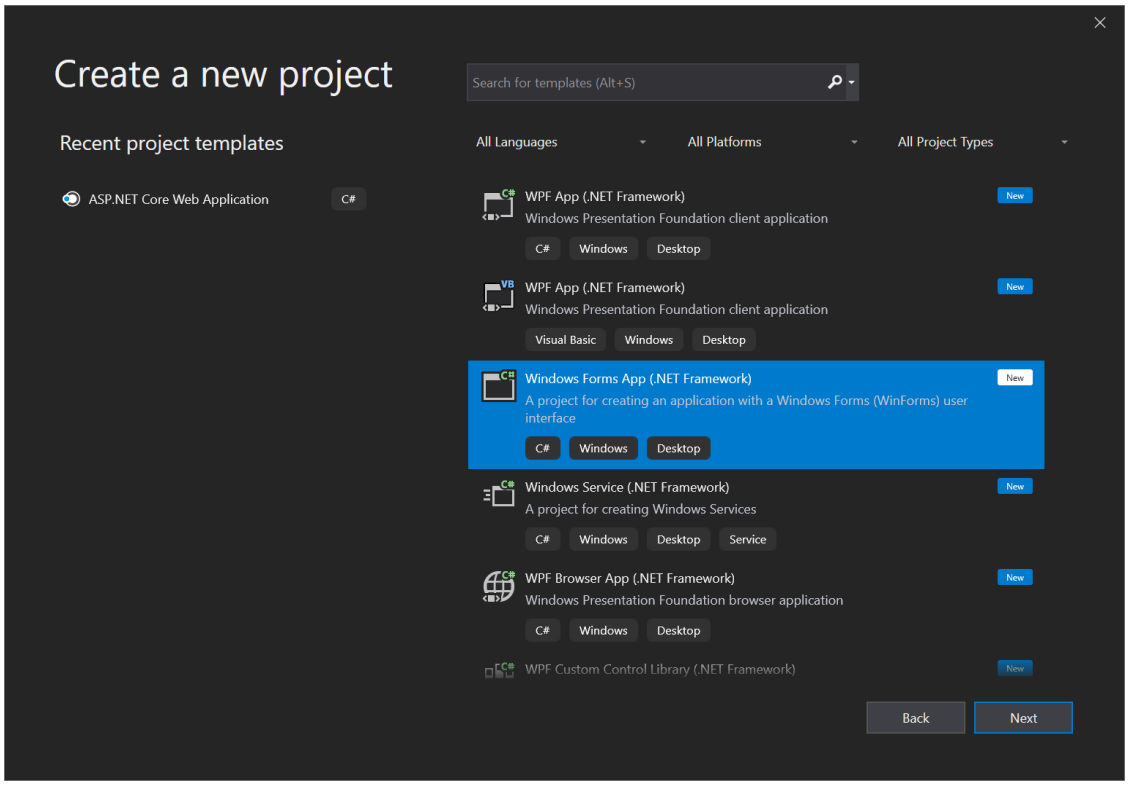

Imagen 5: Elaboración propia

Luego en la ventana de configuración ingresar el nombre del proyecto, de la solución, la ubicación y versión del Framework .Net al igual que si fuese un proyecto de Consola.

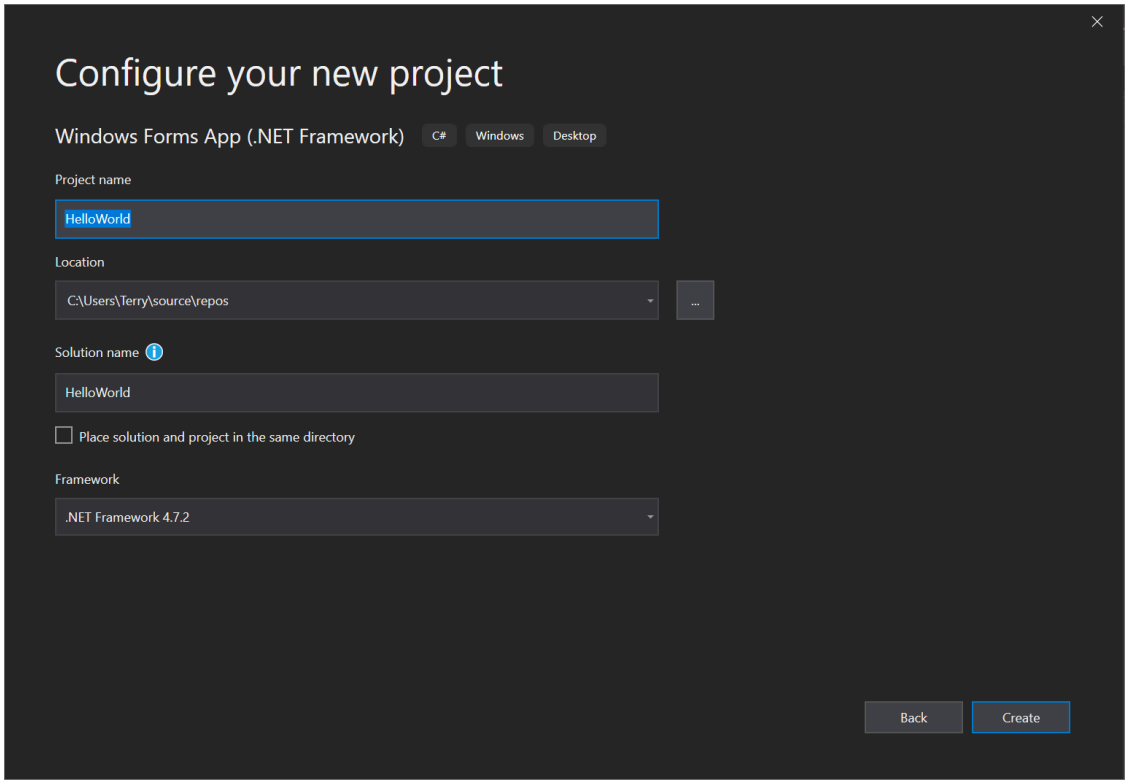

Imagen 6: Elaboración propia

Después de seleccionar su plantilla de proyecto C# y de nombrar su archivo, Visual Studio abre un formulario para usted. Sobre este formulario vamos a agregar dos controles: un botón y una etiqueta. Por último, vamos a agregar el código al formulario para dar respuesta al evento click del botón.

1. Agregar un botón al formulario
    a. Elija Caja de herramientas para abrir la ventana desplegable Caja de herramientas. (Si no ve la opción desplegable Caja de herramientas , puede abrirla desde la barra de menú. Para hacerlo, seleccione Ver > Caja de herramientas . O presione Ctrl + Alt + X).

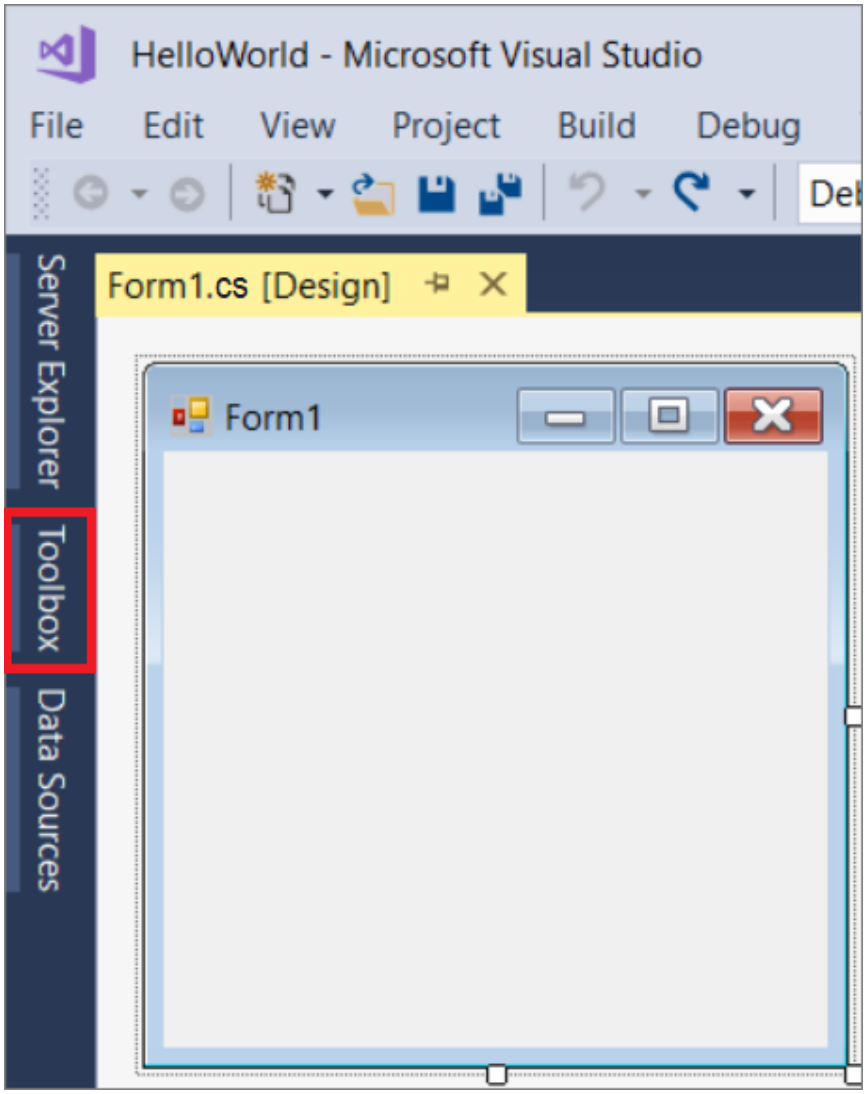

Imagen 7: Elaboración propia

b. Elija el icono de Pin para acoplar la ventana de la Caja de herramientas.

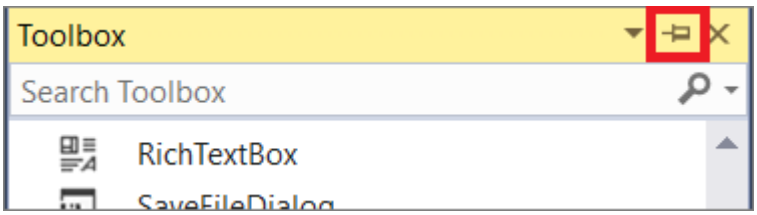

Imagen 8: Elaboración propia

c. Elija el control Botón y luego arrástrelo al formulario.

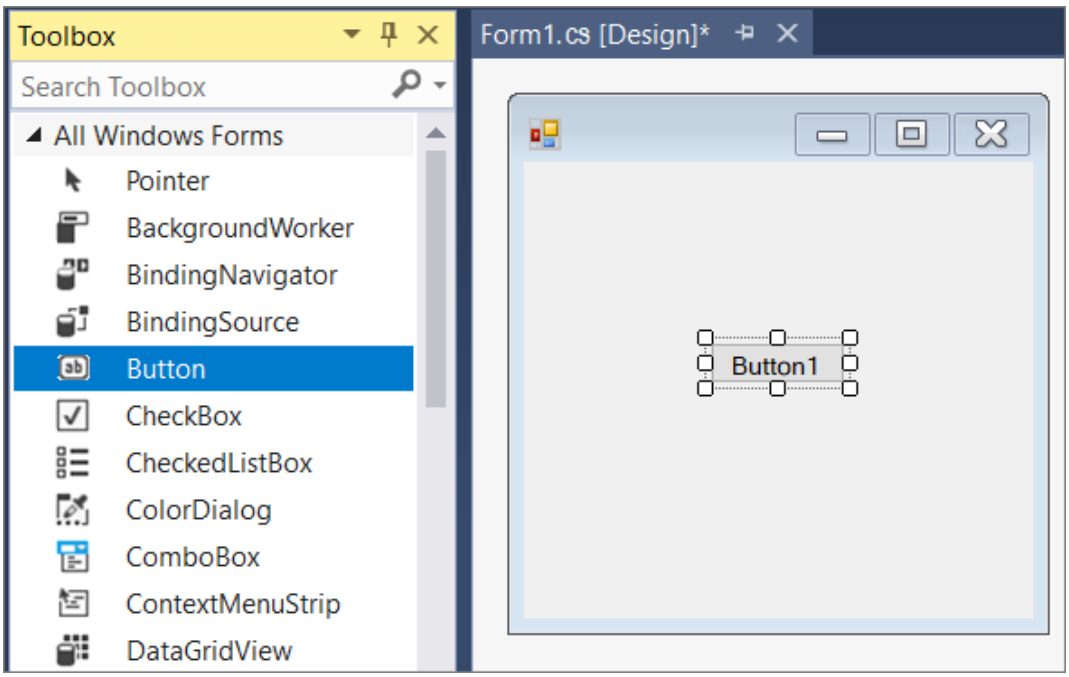

Imagen 9: Elaboración propia

d. En la ventana Propiedades , busque Texto , cambie el nombre de Button1 a `Click this` y luego presione Entrar. (Si no ve la ventana Propiedades , puede abrirla desde la barra de menú. Para hacerlo, elija Ver > Ventana Propiedades . O presione F4).

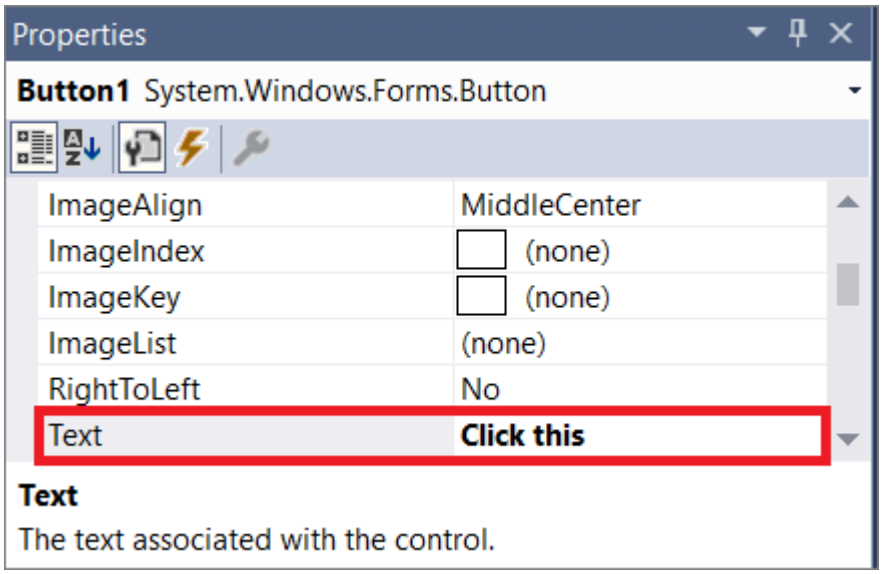

Imagen 10: Elaboración propia

e. En la sección Diseño de la ventana Propiedades , cambie el nombre de Button1 a btnClickThis y luego presione Entrar.

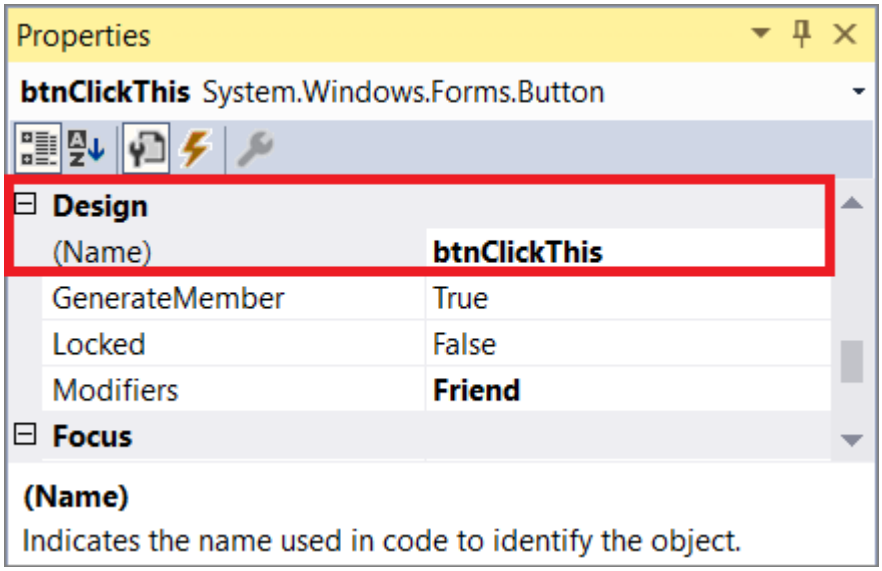

Imagen 11: Elaboración propia

2. Agregar una etiqueta al formulario

Ahora que hemos agregado un control de botón para crear una acción, agreguemos un control de etiqueta para enviar texto.

a. Seleccione el control Etiqueta de la ventana Caja de herramientas y luego arrástrelo al formulario y suéltelo debajo del botón. Haga clic en este.

b. En la sección Diseño o en la sección (Vinculaciones de datos) de la ventana Propiedades, cambie el nombre de Label1 a lblHelloWorld y luego presione Entrar.

3. Agregar código al formulario

a. En la ventana Form1.cs [Diseño] , haga doble clic sobre el botón para abrir la ventana Form1.cs (Como alternativa, puede expandir Form1.cs en el Explorador de soluciones y luego elegir Form1).

b. En la ventana Form1.cs, después de la línea vacía privada, escriba o ingrese `lblHelloWorld.Text = "Hello World!";`como se muestra en la siguiente pantalla:

```
Form1.cs | Form1.cs [Design]
HelloWorld | HelloWorld.Form1
1   using System;
2   using System.Windows.Forms;
3
4   namespace HelloWorld
5   {
6       public partial class Form1 : Form
7       {
8           public Form1()
9           {
10              InitializeComponent();
11          }
12
13          private void btnClickThis_Click(object sender, EventArgs e)
14          {
15              lblHelloWorld.Text = "Hello World!";
16          }
17      }
18  }
19
```

Imagen 12: Elaboración propia

Al ejecutar la aplicación y seleccionar el botón *Click this* obtendremos un resultado similar a este:

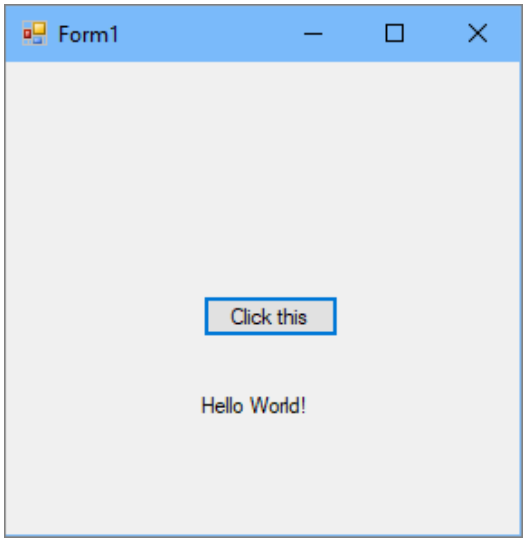

Imagen 13: Elaboración propia

Problema modelo

Calcular el perímetro de un triángulo en una interfaz gráfica.

Solución.

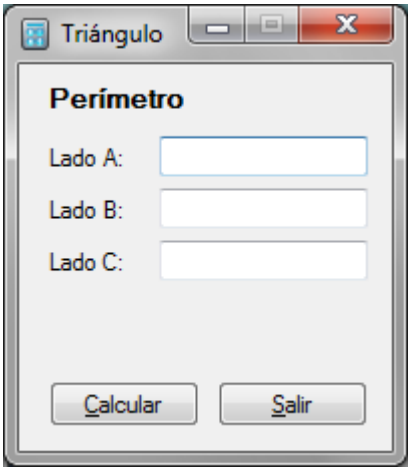

Imagen 14: Elaboración propia

```
namespace ProyectoTriangulo
{
    public partial class frmTriangulo : Form
    {
        public frmTriangulo()
        {
            InitializeComponent();
            lblPerimetro.Visible = false;
        }
        //Clase Triangulo:
        public class Triangulo
        {
            double ladoA, ladoB, ladoC;
            //Constructores:
            public Triangulo()
            {
                ladoA = ladoB = ladoC = 0;
            }
            public Triangulo(float a, float b, float c)
            {
                ladoA = a;
                ladoB = b;
                ladoC = c;
          }
          //Propiedades:
          public double pLadoA
          {
              set { ladoA =   value; }
              get { return ladoA; }
          }
          public double pLadoB
          {
              set { ladoB = value; }
              get { return ladoB; }
          }
```

```
            public double pLadoC
            {
                set { ladoC = value; }
                get { return ladoC; }
            }
            //Métodos:
            public double perimetro()
            {
                return ladoA + ladoB + ladoC;
            }
        }//Fin Clase Triángulo
        //Botón Calcular:
        private void btnCalcular_Click(object sender, EventArgs e)
        {
            Triangulo triang = new Triangulo();
            triang.pLadoA = Convert.ToDouble(txtA.Text);
            triang.pLadoB = Convert.ToDouble(txtB.Text);
            triang.pLadoC = Convert.ToDouble(txtC.Text);
            lblPerimetro.Text = "Perímetro: " + triang.perimetro().ToString();
            lblPerimetro.Visible = true;
            txtA.Text = "";
            txtB.Text = "";
            txtC.Text = "";
            txtA.Focus();
        }//Fin Botón Calcular
        //Botón Salir:
        private void btnSalir_Click(object sender, EventArgs e)
        {
            Close();
        }//Fin Botón Salir
    }
}
```

BIBLIOGRAFÍA

Bishop, P. (1992) Fundamentos de Informática. Anaya.

Brookshear, G. (1994) Computer Sciense: An Overview. Benjamin/Cummings.

De Miguel, P. (1994) Fundamentos de los Computadores. Paraninfo.

Joyanes, L. (1990) Problemas de Metodología de la Programación. McGraw Hill.

Joyanes, L. (1993) Fundamentos de Programación: Algoritmos y Estructura de Datos. McGraw Hill.

Norton, P. (1995) Introducción a la Computación. McGraw Hill.

Prieto, P. Lloris A. y Torres J.C. (1989) Introducción a la Informática. McGraw Hill.

Tucker, A. Bradley, W. Cupper, R y Garnick, D (1994) Fundamentos de Informática (Lógica, resolución de problemas, programas y computadoras). McGraw Hill.

Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Referenciarlo de la siguiente manera: Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.